Université de Limoges
Faculté des Sciences & Techniques
Faculté des Sciences et Techniques de Limoges

L2 - Bases du langage C
TP3 - Manipulation de pointeurs

Maxime MARIA

Respectez bien les consignes et testez le bon fonctionnement de tous vos programmes ! Pensez aussi à sauvegarder régulièrement ainsi qu'à garder tous vos codes !

1 Découverte des pointeurs

1. Dans le programme suivant, nous voulons modifier la valeur de la variable `a` en utilisant un pointeur vers un entier `b`. Ainsi, les deux affichages doivent donner la même chose : 100.
   Le souci, c'est que ce code ne marche pas comme il faut.
   Corrigez-le ! (Vous n'avez aucune ligne à ajouter, il faut juste corriger celles présentes).

```c
#include <stdio.h>

int main(void)
{
    int a = 87;
    int *b = a;
    b += 13;

    printf("a = %d\n", a);
    printf("b = %d\n", b);
}
```

2. Maintenant, nous voulons directement modifier l'entier `a` en utilisant une fonction. Testez le code suivant, la valeur de `a` devrait être 100, mais il ne fonctionne pas. Pourquoi ?
   Corrigez-le ! (Vous n'avez aucune ligne à ajouter, il faut juste corriger celles présentes).

```c
#include <stdio.h>

void plus_treize(int a)
{
    a = a + 13;
}

int main(void)
{
    int a = 87;
    plus_treize(a);
    printf("a = %d\n", a);
}
```

2 Passage d'arguments par valeur ou par adresse

Pour chaque question de cet exercice, vous écrirez le programme principal (`main`) qui vous permettra de vérifier le bon fonctionnement de votre fonction.

1. Écrivez une fonction `void swap(int *a, int *b)` qui échange les valeurs de `a` et `b`.
2. Écrivez une fonction `int gcd(int a, int b)` qui calcule et retourne le plus grand diviseur commun, ou PCGD [1], de `a` et `b` (`gcd` = *Greatest Common Divisor*). Vous ne devez utiliser qu'une seule variable locale à la fonction et utiliser directement les paramètres de la fonction (passés par copie). Vous constaterez que les paramètres transmis depuis le `main` n'ont pas été modifiés.
3. Écrivez une fonction `int simplifyFraction(int *num, int *den)` qui prend en paramètres deux valeurs représentant le numérateur (`num`) et le dénominateur (`den`) d'une fraction, et modifie ces valeurs pour qu'elles correspondent à une fraction équivalente simplifiée. Par exemple, si les variables données à la fonction ont pour valeur 4 et 8, son exécution modifiera ces valeurs de sorte qu'elles soient égales à 1 et 2 ($\frac{4}{8} = \frac{1}{2}$). Si le dénominateur est égal à 0, alors la fonction devra indiquer une erreur en retournant -1 (elle retournera 0 sinon).
   Indice : la fonction `gcd` vous sera utile !

3 Au tableau !

Maintenant que nous savons utiliser les pointeurs sur une seule variable, nous allons voir comment manipuler des tableaux grâces à eux. En effet, un tableau en C est un ensemble de valeurs stockées en mémoire de façon contiguë. La variable qui contient le tableau est un pointeur vers la première case du celui-ci, c'est-à-dire l'adresse du tableau en mémoire.

L'extrait de code suivant permet de créer un tableau de cinq entiers dont les valeurs sont directement données :

```c
int main(void)
{
    int array[5] = {1, 2, 3, 4, 5};
    array[2] += 10;
    printf("%d", array[0]); //Affiche "1", la premiere case du tableau
    printf("%d", array[2]); //Affiche "13", la troisieme case du tableau
                            //apres modification
    printf("%d", array[4]); //Affiche "5", la derniere case du tableau
}
```

Ici, nous avons créé un pointeur constant `array` vers un tableau de cinq entiers. Dans cet exercice, nous allons créer des tableaux dynamiques.

1. Écrivez un programme qui prend en entrée une taille N et crée un tableau dynamiquement alloué de N entiers (`malloc`). Vous remplirez ce tableau avec les entiers de 1 à N. N'oubliez pas de libérer la mémoire à l'aide de la fonction `free`.
2. Écrivez une fonction `displayArray` qui prend en paramètre un pointeur vers un tableau d'entier ainsi que sa taille et affiche son contenu avec un espace entre chaque valeur.
3. Écrivez une fonction `sum` qui calcule et retourne la somme de tous les éléments d'un tableau.
4. Écrivez une fonction `mean` qui calcule et retourne la moyenne de tous les éléments d'un tableau (vous réutiliserez la fonction `sum` précédemment écrite).
5. Écrivez une fonction `mirrorSlice` qui renverse les éléments d'un tableau situés entre deux indices `idMin` inclus et `idMax` exclus passés en paramètres. Cette fonction doit s'exécuter « en place », *i.e.* sans utiliser de tableau auxiliaire. Vous supposerez les indices valides. Vous utiliserez la fonction `swap` écrite à l'exercice précédent.
   Par exemple, si nous avons un tableau `array` contenant `[1, 2, 3, 4, 5]` et que nous appelons la fonction telle que `mirrorSlice(array, 1, 4)`, alors `array` contiendra `[1, 4, 3, 2, 5]`.
6. Écrivez une fonction `copyArray` qui reçoit deux tableaux de même taille (`source` et `destination`) et copie le premier dans le second.

1. https://fr.wikipedia.org/wiki/Plus_grand_commun_diviseur